8-puzzle

Code:

```
from collections import deque


GOAL_STATE = (0, 1, 2,
              3, 4, 5,
              6, 7, 8)


MOVES = {
    0: [1, 3],
    1: [0, 2, 4],
    2: [1, 5],
    3: [0, 4, 6],
    4: [1, 3, 5, 7],
    5: [2, 4, 8],
    6: [3, 7],
    7: [4, 6, 8],
    8: [5, 7]
}


def print_state(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()


def bfs(initial_state):
    queue = deque([(initial_state, [])])
    visited = set()


    while queue:
```

```
        current_state, path = queue.popleft()

        if current_state == GOAL_STATE:
            return path + [current_state]

        visited.add(current_state)
        zero_index = current_state.index(0)

        for move in MOVES[zero_index]:
            new_state = list(current_state)
            new_state[zero_index], new_state[move] = new_state[move], new_state[zero_index]
            new_state = tuple(new_state)

            if new_state not in visited:
                queue.append((new_state, path + [current_state]))

    return None
initial = (1, 2, 3,
          4, 0, 6,
          7, 5, 8)
solution = bfs(initial)
if solution:
    print("Solution found in", len(solution) - 1, "moves\n")
    for step in solution:
        print_state(step)
else:
    print("No solution found.")
```

Output:

Solution found in 20 moves

(1, 2, 3)

(4, 0, 6)

(7, 5, 8)

(1, 2, 3)

(4, 6, 0)

(7, 5, 8)

(1, 2, 0)

(4, 6, 3)

(7, 5, 8)

(1, 0, 2)

(4, 6, 3)

(7, 5, 8)

(0, 1, 2)

(4, 6, 3)

(7, 5, 8)

(4, 1, 2)

(0, 6, 3)

(7, 5, 8)

(4, 1, 2)
(6, 0, 3)
(7, 5, 8)

(4, 1, 2)
(6, 3, 0)
(7, 5, 8)

(4, 1, 0)
(6, 3, 2)
(7, 5, 8)

(4, 0, 1)
(6, 3, 2)
(7, 5, 8)

(4, 3, 1)
(6, 0, 2)
(7, 5, 8)

(4, 3, 1)
(6, 5, 2)
(7, 0, 8)

(4, 3, 1)

(6, 5, 2)

(0, 7, 8)

(4, 3, 1)

(0, 5, 2)

(6, 7, 8)

(0, 3, 1)

(4, 5, 2)

(6, 7, 8)

(3, 0, 1)

(4, 5, 2)

(6, 7, 8)

(3, 1, 0)

(4, 5, 2)

(6, 7, 8)

(3, 1, 2)

(4, 5, 0)

(6, 7, 8)

(3, 1, 2)

(4, 0, 5)

(6, 7, 8)

(3, 1, 2)

(0, 4, 5)

(6, 7, 8)

(0, 1, 2)

(3, 4, 5)

(6, 7, 8)